

React-JSX Stack Detailed Guide

Unit 2: React.js - Component Architecture & JSX

1. What is React.js?

React.js is a JavaScript library for building user interfaces, particularly single-page applications where you need a fast, interactive user experience.

Key Characteristics: - **Component-Based:** UI is built from small, reusable pieces - **Declarative:** Tell React what you want, not how to do it - **Virtual DOM:** Efficient updating mechanism - **Learn Once, Write Anywhere:** Can be used for web, mobile (React Native), desktop

Analogy: Think of React like LEGO blocks - you create small, reusable pieces (components) that can be combined to build complex structures (applications).

2. React Component Architecture

What are Components?

Components are independent, reusable pieces of code that return JSX elements describing what should appear on the screen.

Types of Components:

A. Functional Components (Modern Approach)

```
// Simple functional component
function Greeting() {
  return <h1>Hello, World!</h1>;
}

// Arrow function component
const Greeting = () => {
  return <h1>Hello, World!</h1>;
};

// Component with props
function UserCard({ name, age, email }) {
  return (
    <div className="user-card">
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Email: {email}</p>
    </div>
  );
}
```

B. Class Components (Legacy Approach)

```
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h1>Hello, {this.props.name}!</h1>;
  }
}
```

Component Composition Example:

```
function Header() {
  return (
    <header>
      <h1>My Website</h1>
      <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
      </nav>
    </header>
  );
}

function Footer() {
  return (
    <footer>
      <p>&copy; 2024 My Website. All rights reserved.</p>
    </footer>
  );
}

function App() {
  return (
    <div>
      <Header />
      <main>
        <h2>Welcome to my website!</h2>
        <p>This is the main content area.</p>
      </main>
      <Footer />
    </div>
  );
}
```

3. JSX (JavaScript XML) - Deep Dive

What is JSX?

JSX is a syntax extension for JavaScript that allows you to write HTML-like code inside JavaScript. It makes React components more readable and easier to write.

Behind the Scenes:

```
// JSX
```

```
const element = <h1>Hello, world!</h1>;

// Transpiled JavaScript (what Babel converts JSX to)
const element = React.createElement('h1', null, 'Hello, world!');
```

JSX Syntax Rules:

1. Single Root Element:

```
// ✗ Wrong - Multiple root elements
function App() {
  return (
    <h1>Title</h1>
    <p>Paragraph</p>
  );
}

// ✔ Correct - Single root element
function App() {
  return (
    <div>
      <h1>Title</h1>
      <p>Paragraph</p>
    </div>
  );
}

// ✔ Also correct - React Fragment
function App() {
  return (
    <>
      <h1>Title</h1>
      <p>Paragraph</p>
    </>
  );
}
```

2. Self-Closing Tags:

```
// ✗ Wrong

<input type="text" name="username">
<br>

// ✔ Correct

<input type="text" name="username" />
<br />
```

3. Attribute Name Conversions:

```
// HTML → JSX conversions
class → className
for → htmlFor
tabindex → tabIndex
onclick → onClick
onchange → onChange
```

Embedding JavaScript Expressions:

Variables & Functions:

```
function UserProfile() {
  const userName = "John Doe";
  const userAge = 25;

  const formatAge = (age) => {
    return age >= 18 ? `${age} (Adult)` : `${age} (Minor)`;
  };

  return (
    <div>
      <h1>User: {userName}</h1>
      <p>Age: {formatAge(userAge)}</p>
      <p>Born in: {2024 - userAge}</p>
    </div>
  );
}
```

Conditional Rendering:

```
function LoginButton({ isLoggedIn, userName }) {
  return (
    <div>
      {isLoggedIn ? (
        <div>
          <span>Welcome, {userName}!</span>
          <button>Logout</button>
        </div>
      ) : (
        <button>Login</button>
      )}
    </div>
  );
}

// Logical AND for conditional display
function Notification({ hasMessages, messageCount }) {
  return (
    <div>
      <h1>Dashboard</h1>
      {hasMessages && (
        <div className="notification">
          You have {messageCount} new messages!
        </div>
      )}
    </div>
  );
}
```

Rendering Lists:

```
function TodoList() {
  const todos = [
    { id: 1, text: "Learn React", completed: false },
    { id: 2, text: "Build a project", completed: false },
    { id: 3, text: "Deploy to production", completed: true }
  ];

  return (
    <ul>
```

```

      {todos.map(todo => (
        <li
          key={todo.id}
          style={{
            textDecoration: todo.completed ? 'line-through' : 'none'
          }}
        >
          {todo.text}
        </li>
      ))}
    </ul>
  );
}

```

4. Props (Properties) - Data Flow

What are Props?

Props are how components communicate with each other. They are passed from parent components to child components.

Basic Props Example:

```

// Child component
function ProductCard({ title, price, image, inStock }) {
  return (
    <div className="product-card">
      <img src={image} alt={title} />
      <h3>{title}</h3>
      <p className="price">${price}</p>
      {inStock ? (
        <button className="buy-btn">Buy Now</button>
      ) : (
        <button className="out-of-stock" disabled>
          Out of Stock
        </button>
      )}
    </div>
  );
}

// Parent component
function ProductList() {
  const products = [
    {
      id: 1,
      title: "Laptop",
      price: 999.99,
      image: "/images/laptop.jpg",
      inStock: true
    },
    {
      id: 2,
      title: "Mouse",
      price: 29.99,
      image: "/images/mouse.jpg",
      inStock: false
    }
  ]
}

```

```

];

return (
  <div className="product-list">
    {products.map(product => (
      <ProductCard
        key={product.id}
        title={product.title}
        price={product.price}
        image={product.image}
        inStock={product.inStock}
      />
    ))}
  </div>
);
}

```

Props Destructuring & Default Values:

```

// With destructuring and default values
function UserBadge({
  name = "Anonymous",
  avatar = "/default-avatar.png",
  role = "User",
  isOnline = false
}) {
  return (
    <div className={`user-badge ${isOnline ? 'online' :
'offline'}`}>
      <img src={avatar} alt={`${name}'s avatar`} />
      <div className="user-info">
        <h4>{name}</h4>
        <span className="role">{role}</span>
        <span className={`status ${isOnline ? 'online' :
'offline'}`}>
          {isOnline ? 'Online' : 'Offline'}
        </span>
      </div>
    </div>
  );
}

// Usage
<UserBadge name="Alice" role="Admin" isOnline={true} />
<UserBadge name="Bob" />

```

Children Props:

```

// Modal component that accepts children
function Modal({ title, isOpen, onClose, children }) {
  if (!isOpen) return null;

  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={e =>
e.stopPropagation()}>
        <div className="modal-header">
          <h2>{title}</h2>
          <button onClick={onClose}>×</button>
        </div>
        <div className="modal-body">

```

```

        {children}
      </div>
    </div>
  </div>
);
}

// Usage
function App() {
  const [isModalOpen, setIsModalOpen] = useState(false);

  return (
    <div>
      <button onClick={() => setIsModalOpen(true)}>
        Open Modal
      </button>

      <Modal
        title="Confirm Action"
        isOpen={isModalOpen}
        onClose={() => setIsModalOpen(false)}
      >
        <p>Are you sure you want to delete this item?</p>
        <div className="modal-actions">
          <button onClick={() =>
setIsModalOpen(false)}>Cancel</button>
          <button className="danger">Delete</button>
        </div>
      </Modal>
    </div>
  );
}

```

5. Event Handling in React

Synthetic Events:

React wraps native events in SyntheticEvent objects that work consistently across browsers.

```

function InteractiveForm() {
  const handleSubmit = (event) => {
    event.preventDefault(); // Prevent default form submission
    console.log('Form submitted!');
  };

  const handleInputChange = (event) => {
    console.log('Input value:', event.target.value);
  };

  const handleButtonClick = (event, customData) => {
    console.log('Button clicked!', customData);
    console.log('Event type:', event.type);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input

```

```

        type="text"
        onChange={handleInputChange}
        placeholder="Type something..."
      />

      <button
        type="button"
        onClick={e => handleButtonClick(e, { id: 123, name: 'Test'
    }}}

    >
      Click Me
    </button>

    <button type="submit">Submit</button>
  </form>
);
}

```

Event Handler Patterns:

```

function TodoApp() {
  const [todos, setTodos] = useState([
    { id: 1, text: "Learn React", completed: false }
  ]);

  // Method 1: Inline arrow function
  const handleToggle1 = (id) => {
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } :
    todo
    ));
  };

  // Method 2: Curry function for performance
  const handleToggle2 = (id) => (event) => {
    console.log('Toggled todo with ID:', id);
    setTodos(todos.map(todo =>
      todo.id === id ? { ...todo, completed: !todo.completed } :
    todo
    ));
  };

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          <span>{todo.text}</span>

          /* Method 1: Inline arrow function */
          <button onClick={() => handleToggle1(todo.id)}>
            Toggle (Method 1)
          </button>

          /* Method 2: Pre-bound function */
          <button onClick={handleToggle2(todo.id)}>
            Toggle (Method 2)
          </button>
        </li>
      ))}
    </ul>
  );
}

```

```
}
```

6. Virtual DOM Concept

What is Virtual DOM?

The Virtual DOM is a JavaScript representation of the real DOM kept in memory. React uses it to optimize updates.

How it Works: 1. **Initial Render:** React creates a virtual DOM tree 2. **State Change:** React creates a new virtual DOM tree 3. **Diffing:** React compares (diffs) old and new trees 4. **Reconciliation:** React updates only changed parts in real DOM

Example of Virtual DOM Benefit:

```
function CounterExample() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState("John");

  // When count changes, only the count display updates
  // The name input stays unchanged
  return (
    <div>
      <h1>Counter App</h1>
      <p>Count: {count}</p> /* Only this updates */
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>

      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        placeholder="Enter name"
      />
      <p>Hello, {name}!</p>
    </div>
  );
}
```

7. React Rendering Process

Element Rendering:

```
// Creating React elements
const element1 = React.createElement('h1', null, 'Hello World');
const element2 = <h1>Hello World</h1>; // JSX version

// Rendering to DOM (React 18+)
import { createRoot } from 'react-dom/client';

const container = document.getElementById('root');
const root = createRoot(container);

// Simple element
```

```
root.render(<h1>Hello, world!</h1>);
```

```
// Component
```

```
root.render(<App />);
```

Conditional Rendering Patterns:

```
function Dashboard({ user, isLoading, error }) {  
  // Early return pattern  
  if (isLoading) {  
    return <div className="loading">Loading...</div>;  
  }  
  
  if (error) {  
    return (  
      <div className="error">  
        <h2>Error occurred!</h2>  
        <p>{error.message}</p>  
      </div>  
    );  
  }  
  
  if (!user) {  
    return <div className="no-user">Please log in to continue.  
</div>;  
  }  
  
  // Main content  
  return (  
    <div className="dashboard">  
      <h1>Welcome, {user.name}!</h1>  
      <div className="user-stats">  
        <div>Posts: {user.posts}</div>  
        <div>Followers: {user.followers}</div>  
        <div>Following: {user.following}</div>  
      </div>  
    </div>  
  );  
}
```

8. Best Practices & Common Patterns

Component Organization:

```
// Good: Small, focused components
```

```
function Avatar({ src, alt, size = 'medium' }) {  
  return (  
    <img  
      src={src}  
      alt={alt}  
      className={`avatar avatar--${size}`}  
    />  
  );  
}
```

```
function UserInfo({ name, email }) {  
  return (  
    <div className="user-info">
```

```

        <h3>{name}</h3>
        <p>{email}</p>
      </div>
    );
  }

  function UserCard({ user }) {
    return (
      <div className="user-card">
        <Avatar
          src={user.avatar}
          alt={`${user.name}'s avatar`}
          size="large"
        />
        <UserInfo name={user.name} email={user.email} />
      </div>
    );
  }

```

Key Props for Lists:

```

// ✗ Bad: No keys or index as key
function BadList({ items }) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index}>{item.name}</li> // Don't use index
      ))}
    </ul>
  );
}

// ✔ Good: Stable, unique keys
function GoodList({ items }) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li> // Use unique ID
      ))}
    </ul>
  );
}

```

Performance Tips:

```

// Memoization for expensive calculations
import { useMemo } from 'react';

function ExpensiveComponent({ items, filter }) {
  const filteredItems = useMemo(() => {
    return items.filter(item =>
      item.name.toLowerCase().includes(filter.toLowerCase())
    ).sort((a, b) => a.name.localeCompare(b.name));
  }, [items, filter]);

  return (
    <div>
      {filteredItems.map(item => (
        <div key={item.id}>{item.name}</div>

```

```

    )})
  </div>
);
}

```

9. Common JSX Gotchas & Solutions

Styling with Dynamic Classes:

```

function Button({ variant, size, disabled, children }) {
  // Method 1: Template literals
  const className = `btn btn--${variant} btn--${size} ${disabled ?
'btn--disabled' : ''}`;

  // Method 2: Array join
  const classNames = [
    'btn',
    `btn--${variant}`,
    `btn--${size}`,
    disabled && 'btn--disabled'
  ].filter(Boolean).join(' ');

  return (
    <button className={classNames} disabled={disabled}>
      {children}
    </button>
  );
}

```

Handling Forms:

```

function ContactForm() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    message: ''
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData(prev => ({
      ...prev,
      [name]: value
    }));
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    console.log('Form data:', formData);
    // Submit logic here
  };

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        name="name"
        value={formData.name}

```

```

        onChange={handleChange}
        placeholder="Name"
        required
      />
      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="Email"
        required
      />
      <textarea
        name="message"
        value={formData.message}
        onChange={handleChange}
        placeholder="Message"
        required
      />
      <button type="submit">Send Message</button>
    </form>
  );
}

```

Summary Table

Concept	What It Means	Example Use Case
Component	Reusable UI building block	Button, Header, UserCard
JSX	HTML-like syntax in JavaScript	Writing component templates
Props	Data passed to components	Customizing component behavior
Virtual DOM	In-memory representation of DOM	Efficient UI updates
Event Handling	Responding to user interactions	Click, form submission, input
Conditional Rendering	Show/hide based on conditions	Login/logout states, loading

Key Takeaways:

- **Components are functions** that return JSX
- **Props flow down**, events flow up
- **JSX looks like HTML** but has JavaScript superpowers
- **Virtual DOM makes React fast** by minimizing actual DOM updates
- **Always use keys** when rendering lists
- **Keep components small** and focused on single responsibility